

TRACE EVALUATION REPORT

TRACE Evaluation

SARMAAN LC Dashboard

SARMAAN LGA Coordinator (LC) Dashboard

eHealth Africa · SARMAAN / Kano AMR

Document Version	1.0
Date	August 2026
Repository	sarmaan_lc_dashboard
Branch	feature/trace-evaluation
Commit Evaluated	484e2d9
Framework	TRACE (Technical / Real-world / Adaptive / Outcomes / Economic)
Evaluated By	Joshua Ogundairo (with Claude Code)
Overall Score	4.59 / 10
Status	Final

This report documents the design, scope, and results of a TRACE evaluation of The SARMAAN LGA Coordinator (LC) Dashboard – a zero-infrastructure static site plus an hourly GitHub Actions KoboToolbox data pipeline – as a software product, including its actual current live operational state, not only its designed intent. It records what was scored, on what evidence, what was found, and what remains open for follow-up.

Contents

1	Executive Summary	2
2	Framework and Methodology	3
2.1	Technical (20%)	3
2.2	Real-world (25%)	3
2.3	Adaptive (20%)	3
2.4	Outcomes (25%)	4
2.5	Economic (10%)	4
3	Results	5
3.1	Scorecard	5
3.2	Criterion-Level Detail	7
3.3	Technical — 5.33 / 10	7
3.4	Real-world — 5.67 / 10	7
3.5	Adaptive — 3.00 / 10	8
3.6	Outcomes — 3.50 / 10	8
3.7	Economic — 6.33 / 10	9
4	Key Findings and Interpretation	10
4.1	Strengths	10
4.2	Open Issues	10
5	Limitations of This Evaluation	12
6	Reproducibility	13

1. Executive Summary

This report applies the TRACE Evaluation Framework to the SARMAAN LGA Coordinator (LC) Dashboard — a deliberately zero-infrastructure static site paired with an hourly GitHub Actions pipeline that pulls field-submission data from KoboToolbox — as it stands on `main` (commit `484e2d9`). TRACE scores a complex intervention across five weighted pillars — Technical, Real-world, Adaptive, Outcomes, Economic — looking at the whole trajectory (the *how*) alongside the final result (the *what*), not a single-point metric.

This evaluation surfaced a live, currently-ongoing operational failure, not just a static code review: the repository’s own automated data pipeline has failed 100 out of its last 100 recorded runs (verified via `gh run list`), with the specific error `KOBO_TOKEN environment variable not set` (verified via `gh run view --log-failed`), and GitHub Pages is not even enabled for this repository (confirmed via the GitHub API). The scores below reflect the platform’s *actual current state*, not only its designed intent — this is a deliberate and, per this report’s framing, necessary departure from evaluating code quality in isolation.

Key Numbers at a Glance

- **Overall TRACE score: 4.59 / 10** across 5 pillars and 15 criteria — the lowest score in this evaluation batch, driven almost entirely by a currently-broken production deployment rather than by code quality
- **The system’s one automated job has a 0% success rate:** 100 of the last 100 Fetch KoboToolbox Data workflow runs have failed with `KOBO_TOKEN environment variable not set` — a one-line configuration fix (adding the missing GitHub Actions secret), not a code defect
- **Data has been frozen for over 5 weeks:** `data.json` is stamped `fetches_at: 2026-07-07 01:05 WAT`, more than five weeks stale as of this evaluation (2026-08-11)
- **GitHub Pages is not enabled for this repository** (confirmed via `gh api repos/.../pages` returning 404) — there is currently no live URL serving this dashboard at all, despite the README’s detailed deployment guide
- **Strongest pillar: Economic (6.33 / 10)** — the underlying ‘zero-infrastructure’ architecture (no server, no database, no hosting bill) is genuinely well-designed and remains sound even while the pipeline is down

2. Framework and Methodology

TRACE evaluates an intervention across five weighted pillars. Each pillar is scored as the mean of its own criteria (each criterion scored 0–10 against cited evidence); the overall score is the weight-weighted mean of the five pillar scores, also on a 0–10 scale. A criterion score without a written justification is rejected by the scoring tool — every number in this report traces back to a specific, citable piece of evidence.

Table 1. TRACE pillars and weights

Pillar	Weight	What it measures
Technical	20%	Functional, structural, and foundational robustness of the intervention.
Real-world	25%	Performance in authentic, noisy environments rather than controlled, ideal settings.
Adaptive	20%	The capacity to learn, change, and improve based on feedback.
Outcomes	25%	The final results, evaluating whether the intended goals were achieved effectively.
Economic	10%	The financial viability and efficiency of the intervention.

Scoring scale: 0–10 per criterion. ≥ 7.0 : strong, evidenced, no material gap. **5.0–6.9**: adequate, with a specific, named gap. < 5.0 : weak — an open risk or unmeasured area that should be treated as a priority action.

Criteria by Pillar

2.1 Technical (20%)

Functional, structural, and foundational robustness of the intervention.

- **System Performance** — The technical ability to navigate information, use tools, and generate accurate results.
- **Evidence Grounding** — Outputs are supported by factual evidence — preventing hallucinated or unreliable reasoning.
- **Process Efficiency** — The ability to achieve goals with minimal, necessary actions rather than redundant, wasteful ones.

2.2 Real-world (25%)

Performance in authentic, noisy environments rather than controlled, ideal settings.

- **Realism & Robustness** — How well the system performs against unexpected challenges, information traps, and messy/incorrect data.
- **Context Applicability** — How the tool adapts to diverse, complex organizational or social settings.
- **Safety & Reliability** — Whether, in practice, the system behaves in a safe and controllable manner.

2.3 Adaptive (20%)

The capacity to learn, change, and improve based on feedback.

- **Self-Correction** — Ability to recover from errors (recovery latency).
- **Iterative Design** — A 'living document' approach where methodologies are updated based on ongoing monitoring.
- **Feedback Loops** — Utilizing feedback — natural language feedback or performance scores — to refine strategies dynamically.

2.4 Outcomes (25%)

The final results, evaluating whether the intended goals were achieved effectively.

- **Final Success** — The accuracy of the final answer, product, or policy outcome.
- **Impact Analysis** — The difference the intervention made — e.g. reducing cost/risk, improving productivity, changing behavior.
- **Process-Outcome Link** — How effectively the intermediate steps (trajectory) led to the final outcome.

2.5 Economic (10%)

The financial viability and efficiency of the intervention.

- **Cost-Benefit Analysis** — Whether the benefits of the intervention outweigh the costs of implementation.
- **Efficiency Analysis** — How well resources are used (cost-effectiveness).
- **Scalability** — Whether the solution can be scaled sustainably without prohibitive expenses.

3. Results

3.1 Scorecard

Table 2. TRACE pillar scorecard

Pillar	Weight	Score	Rating
Technical	20%	5.33	Adequate, with gaps
Real-world	25%	5.67	Adequate, with gaps
Adaptive	20%	3.00	Weak
Outcomes	25%	3.50	Weak
Economic	10%	6.33	Adequate, with gaps
Overall (weighted)		4.59	Weak

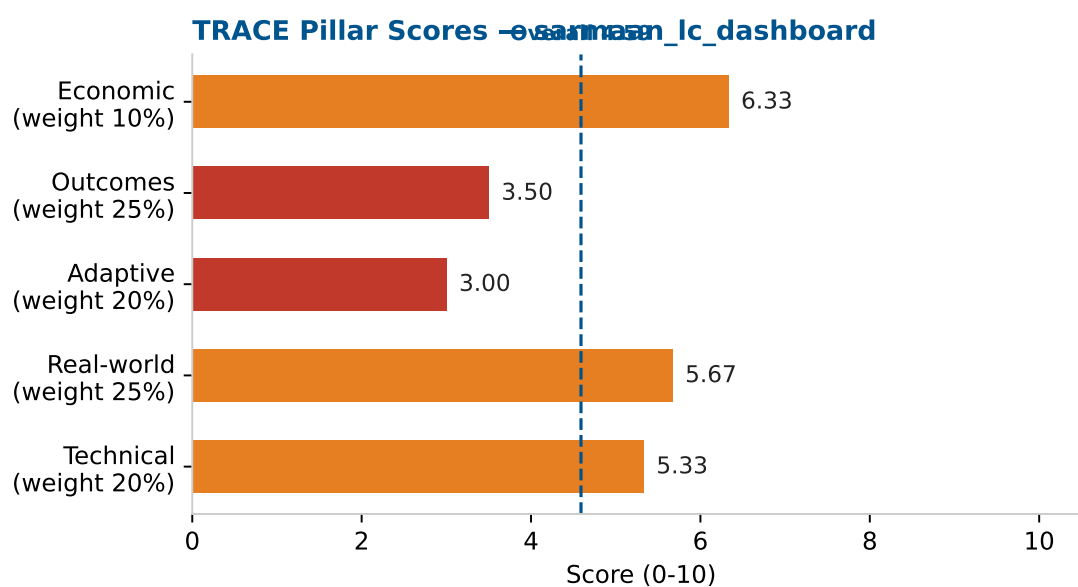


Figure 1. Weighted score per pillar against the overall score (dashed line).

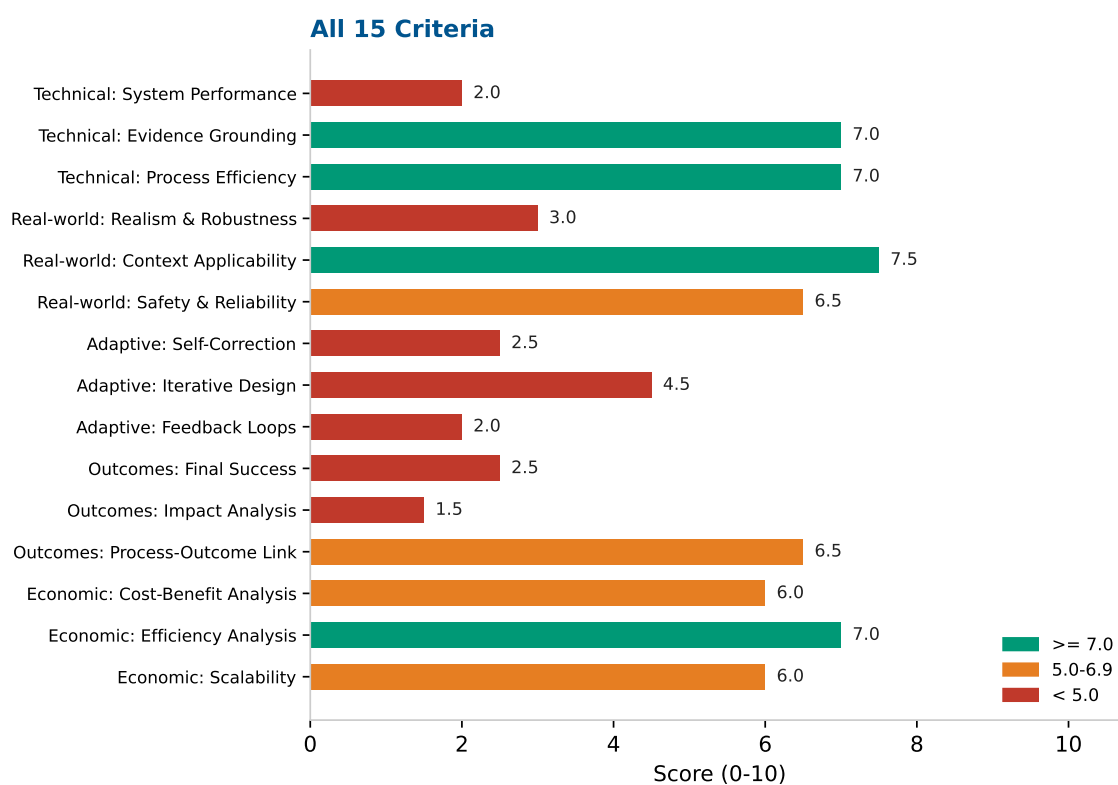


Figure 2. All 15 individual criterion scores, color-banded by rating.

3.2 Criterion-Level Detail

Each box below states the criterion's score and the specific evidence it is grounded in. Colour follows the same scale as Table 2 (green ≥ 7.0 , amber 5.0–6.9, red < 5.0).

3.3 Technical — 5.33 / 10

System Performance — 2.0 / 10

The system's one automated job – hourly fetch of KoboToolbox data via GitHub Actions – has failed 100 out of the last 100 recorded runs (verified via 'gh run list'), with the specific, exact error 'ERROR: KOBO_TOKEN environment variable not set' (verified via 'gh run view -log-failed'). 'data.json' is frozen at 'fetched_at: 2026-07-07 01:05 WAT' with 742 rows, more than five weeks stale as of this evaluation's date (2026-08-11). This is a complete, currently-ongoing operational failure of the system's entire reason for existing, not a hypothetical or historical risk.

Evidence Grounding — 7.0 / 10

fetch_data.py's 'clean(row)' function and its 'g(row, key)' helper are compact and well-scoped, and README.md explicitly documents the exact shape and meaning of every field the frontend consumes (date, coord, lga, status, activity flags, DC roster counts, issue flags). The README's own troubleshooting guidance states the dashboard 'never mutates the file,' so any numeric mismatch a user sees is traceable directly back to the raw Kobo data – a genuinely well-grounded, if small, data contract.

Process Efficiency — 7.0 / 10

Deliberately minimal-dependency design (Python 3 standard library only for the fetch script, no frontend build step, Chart.js loaded from a CDN) means there is very little to run inefficiently. Each hourly workflow run only commits when the fetched data actually changed ('git diff -cached -quiet — git commit ...'), avoiding no-op commit noise on unchanged data.

3.4 Real-world — 5.67 / 10

Realism & Robustness — 3.0 / 10

The one deliberate robustness feature – a coloured freshness status dot (green/amber/red) meant to surface a fetch failure directly to end users – is a sound design idea, but this evaluation found the system has been in the state that dot is meant to flag (fetch failed) continuously for over five weeks, and nothing beyond that in-page visual indicator (no alerting, no notification to a maintainer) surfaces the failure to anyone not actively looking at the page. The safety mechanism exists but has not, in practice, triggered a fix.

Context Applicability — 7.5 / 10

Purpose-built for the real role structure of the SARMAAN/Kano AMR programme: LGA Coordinators reporting daily field activity, with domain-specific fields matching how this specific programme is organized – geofence in/out-of-LGA status with collected reasons, a Data Collector roster (present/partially present/absent), and

an activity-type breakdown (training, field coordination, supervision, stakeholder engagement, problem solving/escalations) that mirrors the coordinators' actual job categories rather than a generic activity log.

Safety & Reliability — 6.5 / 10

Explicit, honest security design: the Kobo API token lives only in a GitHub Actions secret and README.md confirms it 'never appears in any committed file'; 'data.json' is deliberately scoped to non-PII fields with a direct instruction to 'review new fields before adding them to clean(row)'; the README is unusually candid about the public-repo tradeoff required for free GitHub Pages ('treat everything in data.json as world-readable... If you need to hide fields, either drop them in clean(row) or move to a private repo'). Held down by a real, concrete gap: the one secret this entire design depends on ('KOBO_TOKEN') was never actually configured on this specific repository (see System Performance), so the documented security posture and the actual configured state have drifted apart.

3.5 Adaptive — 3.00 / 10

Self-Correction — 2.5 / 10

Only 2 real, human-authored commits exist beyond the 373 automated hourly data-refresh commits (a README expansion, and a UI/UX standards pass) – no bug-fix commit is visible anywhere in the history. Most importantly, the one concrete defect this evaluation found (the missing 'KOBO_TOKEN' secret causing a 100% pipeline failure rate) has been present and unaddressed for over a month despite being trivially visible on the repository's own Actions tab.

Iterative Design — 4.5 / 10

README.md is a genuinely well-organized, thorough living document for the code's size – an architecture diagram, a configuration reference, a troubleshooting table, and a dedicated security-notes section. But it describes an intended steady state rather than showing evidence of active iteration: with only 2 real commits and an ongoing, unaddressed operational failure, there is little sign of this documentation (or the underlying system) being actively kept in sync with the platform's actual current state.

Feedback Loops — 2.0 / 10

No evidence of any feedback loop, external or internal, appears in this repository's history. The clearest possible internal feedback signal available – a GitHub Actions workflow failing on every single run, visible in one click from the repository's Actions tab – does not appear to have been acted on for over a month based on the commit history and the current 100% failure rate.

3.6 Outcomes — 3.50 / 10

Final Success — 2.5 / 10

As designed, this dashboard would competently answer the operational questions it targets. As actually operated right now, it is not succeeding at its core job: GitHub

Pages is not even enabled for this repository (confirmed via ‘gh api repos/.../pages’ returning 404), and the data pipeline has failed continuously for five-plus weeks. Whatever success this system was delivering ended with its last successful fetch on 2026-07-07.

Impact Analysis — 1.5 / 10

The lowest Impact Analysis score in this evaluation batch, reflecting a compounded gap rather than a single one: like most projects evaluated, no instrumentation measures whether this dashboard has changed real programme outcomes – but here that absence is compounded by the fact that, as of this evaluation, the dashboard is not even live (no GitHub Pages site enabled) and has not shown current data to anyone in over five weeks.

Process-Outcome Link — 6.5 / 10

When the pipeline was last running (through 2026-07-07), the link from raw Kobo submissions through ‘clean(row)’ to the specific KPIs and charts shown on the dashboard was clear, traceable, and explicitly documented field-by-field in README.md. This part of the underlying design remains sound in principle even though the pipeline producing the data is not currently running.

3.7 Economic — 6.33 / 10

Cost-Benefit Analysis — 6.0 / 10

The ‘zero-infrastructure’ architecture (README.md’s own framing: ‘no server, no database, no hosting bill’) is a genuinely excellent, deliberately minimal-cost design – static hosting plus a scheduled script, nothing more. The benefit (daily operational visibility for field coordinators) is clear and the marginal cost is close to zero when the pipeline is actually running. Undercut by the fact that even a zero-cost design still requires a minimum of ongoing attention to notice and fix a broken secret, and that attention evidently has not been given for over a month.

Efficiency Analysis — 7.0 / 10

Genuinely efficient by design, independent of its current broken operational state: a standard-library-only fetch script, static file hosting with no server process to keep running, a CDN-served charting library requiring no build step, and no-op commits skipped when fetched data hasn’t changed.

Scalability — 6.0 / 10

The zero-infrastructure design scales close to perfectly for read traffic, since GitHub Pages serves static files. The single-committer-per-hour data model and a single hardcoded ‘ASSET_UID’ scope the architecture to exactly one Kobo form/programme at a time – a reasonable, intentional design boundary for this specific tool’s stated purpose, not a limitation that has emerged from unanticipated growth.

4. Key Findings and Interpretation

4.1 Strengths

Finding 1: The system’s design is genuinely sound where it was actually exercised: a standard-library-only Python fetch script, static GitHub Pages hosting with no server process to keep alive, and a workflow that only commits when fetched data actually changed — a deliberately minimal, low-cost architecture that the README itself frames accurately (“no server, no database, no hosting bill”).

Finding 2: README.md is an unusually thorough living document for a project this size: a full architecture diagram, a field-by-field data-pipeline reference, a configuration reference, a troubleshooting table, and a dedicated, honest security-notes section explicitly stating that everything in `data.json` must be treated as world-readable because the repository has to be public for free GitHub Pages.

Finding 3: The token-handling design is sound in principle: the KoboToolbox API token lives only in an encrypted GitHub Actions secret and is never written into `data.json`, a committed file, or a workflow log.

Finding 4: The specific failure this evaluation found is exactly the failure mode the project’s own README.md Troubleshooting table already anticipates (`KOBO_TOKEN` missing or not configured) — the team clearly understood this risk in the abstract; it simply was not caught in practice on this specific repository.

4.2 Open Issues

Issue 1 [Technical / System Performance]: The Fetch KoboToolbox Data GitHub Actions workflow has failed on every one of its last 100 recorded runs with `ERROR: KOBO_TOKEN environment variable not set` — the required repository secret was never configured on this specific repo. `data.json` has not refreshed since 2026-07-07.

Mitigation: Add the `KOBO_TOKEN` secret under Settings → Secrets and variables → Actions, exactly as README.md’s own deployment guide (§2) already instructs, then manually trigger the workflow once from the Actions tab to confirm the fix before waiting for the next hourly slot.

Issue 2 [Outcomes / Final Success]: GitHub Pages is not enabled for this repository, so there is currently no live URL serving this dashboard to end users at all, independent of the data-freshness issue above.

Mitigation: Follow README.md’s own §6 deployment steps to enable GitHub Pages (Settings → Pages → Deploy from a branch), and resolve the noted `/public` vs. repository-root path mismatch the README itself flags between where the workflow writes `data.json` and where Pages is configured to serve from.

Issue 3 [Adaptive / Feedback Loops]: The clearest possible internal feedback signal available for this system — a workflow failing on every single scheduled run, visible in one click from the repository’s own Actions tab — does not appear to have been acted on for over five weeks.

Mitigation: Beyond fixing the immediate secret, consider adding a low-effort monitor: GitHub can email repository admins on scheduled-workflow failure by default for private repos, or a simple secondary workflow could open/update a pinned issue when the fetch job fails N times in a row, so a stale-data incident like this one surfaces faster next time.

5. Limitations of This Evaluation

1. This is a single-evaluator, qualitative assessment: every 0–10 criterion score is a judgment call grounded in cited evidence (file:line references, commit hashes, or live GitHub Actions run logs), not an automated metric. A second evaluator scoring the same evidence could reasonably land on different numbers within roughly ± 1 point per criterion.
2. Unlike every other project evaluated in this TRACE batch, several scores here (System Performance, Final Success, Impact Analysis) are driven primarily by a live operational check performed during this evaluation session (`gh run list`, `gh run view --log-failed`, `gh api repos/.../pages`) rather than by reading source code alone — these findings reflect the platform’s state at evaluation time (2026-08-11) and could change immediately once the missing secret is added.
3. This evaluation did not have write access to add the missing `KOBO_TOKEN` secret or verify the fix; the recommended mitigation in the Issues section above is inferred from the error message and the README’s own troubleshooting guidance, not independently tested.
4. Real-world and Safety & Reliability scores lean partly on the soundness of the system’s design *as written*, which this evaluation judges separately from its current broken operational state — a reader should not conclude the code itself is poor quality; the gap here is specifically a missing deployment-configuration step.
5. TRACE was originally framed around evaluating AI agent behavior in a single task trajectory. Several sub-criteria (Self-Correction, Iterative Design, Evidence Grounding) were reinterpreted for a software-product context, consistent with the reinterpretation already established in the framework’s earlier applications and carried forward here for comparability across projects.

6. Reproducibility

```
Framework: trace_evaluation/  
Scores file: trace_evaluation/assessments/sarmaan_lc_dashboard/scores.json  
Score command: python -m trace_evaluation.framework.scorer  
sarmaan_lc_dashboard  
Figures command: python trace_evaluation/scripts/generate_figures.py  
sarmaan_lc_dashboard  
Report command: python trace_evaluation/scripts/generate_report.py  
sarmaan_lc_dashboard
```

The scorer and figure generator are fully generic — re-running them against an updated scores.json (after adding the missing KOBO_TOKEN secret and enabling GitHub Pages) reproduces every number and figure in this report deterministically. A re-evaluation after those two fixes is expected to move System Performance, Real-world, Final Success, and Impact Analysis materially upward, since most of what held them down here was a missing deployment step, not a code defect. No external configuration or network access is required to re-run the scorer itself.